

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Чувашский государственный университет им. И.Н. Ульянова»

Кафедра вычислительной техники

Лабораторная работа № 1

По дисциплине “Защита информации”

Выполнил: студент группы ИВТ-42-23

Иванов И.Л.

Проверил: кандидат технических наук

Ковалев С.В.

Чебоксары, 2026

1. Цель работы:

знакомство с классическим криптографическим алгоритмом - алгоритмом шифрования данных при помощи подстановки.

2. Индивидуальное задание.

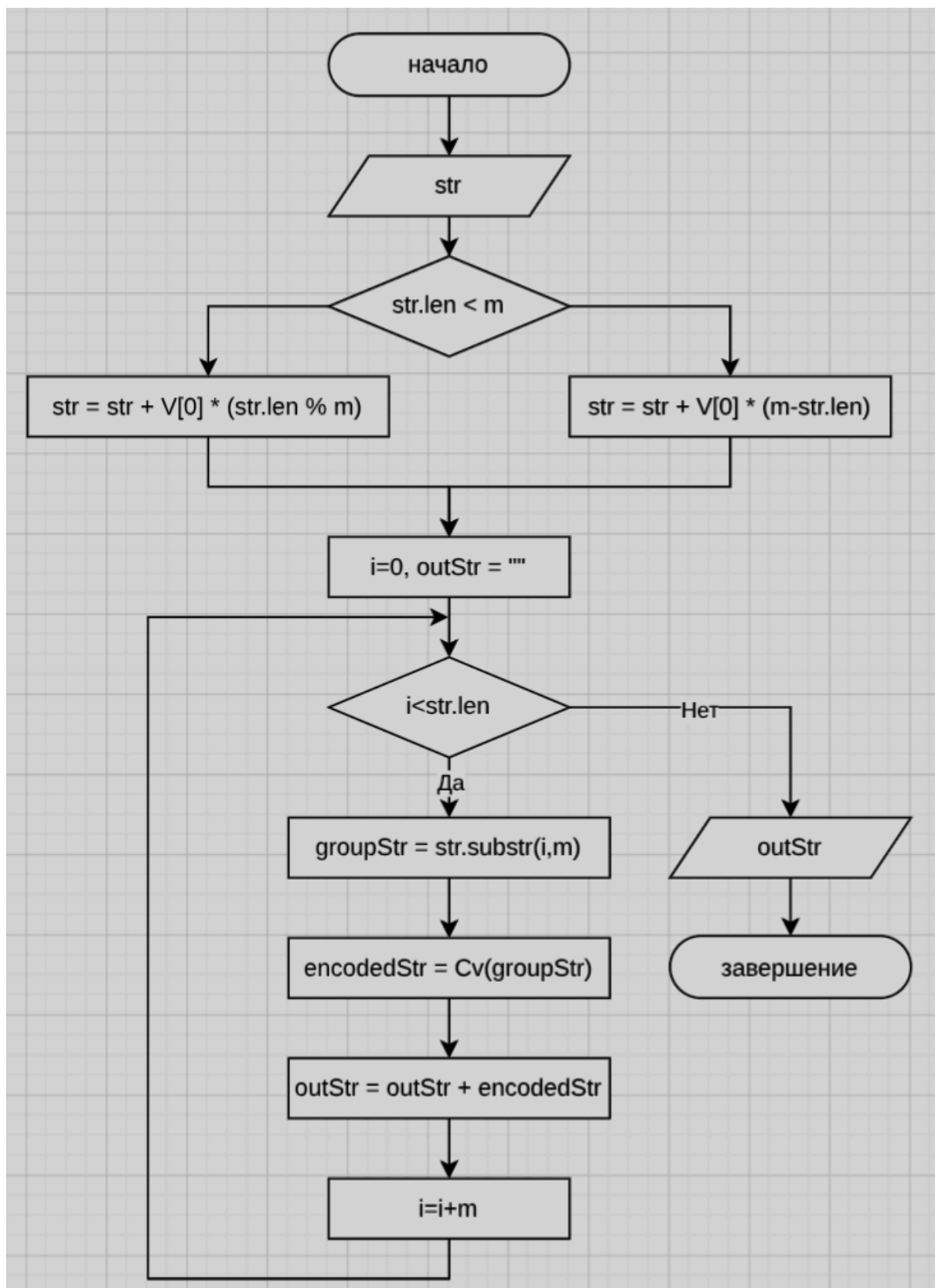
Задание			Для информации	
№ п.п.	V	m		
1	$\{0,1\}$	5	2	32

3. Функция шифрования.

x_i	$C_v(x_i)$
00000	11111
00001	11110
00010	11101
00011	11100
00100	11011
00101	11010
00110	11001
00111	11000
01000	10111
01001	10110
01010	10101
01011	10100
01100	10011
01101	10010
01110	10001
01111	10000
10000	01111

10001	01110
10010	01101
10011	01100
10100	01011
10101	01010
10110	01001
10111	01000
11000	00111
11001	00110
11010	00101
11011	00100
11100	00011
11101	00010
11110	00001
11111	00000

4. Текст программы, реализующей индивидуальное задание.



<https://github.com/kukirmash/labs/tree/main/7sem/ZI/lab1>

```
#include "stdio.h"
```

```
#include "time.h"
```

```
#include <string>
```

```
#include <map>
```

```
#define M 5
```

```
#define N 2
```

```
#define MAX_LEN 19
```

```
#define TEST_COUNT 10
```

```
const char V[N] = {'0', '1'};
```

```
std::map<std::string, std::string> encodeTable{
```

```
    {"00000", "11111"},
```

```
    {"00001", "11110"},
```

```
    {"00010", "11101"},
```

```
    {"00011", "11100"},
```

```
    {"00100", "11011"},
```

```
    {"00101", "11010"},
```

```
    {"00110", "11001"},
```

```
    {"00111", "11000"},
```

```
    {"01000", "10111"},
```

```
    {"01001", "10110"},
```

```
    {"01010", "10101"},
```

```
    {"01011", "10100"},
```

```
    {"01100", "10011"},
```

```
    {"01101", "10010"},
```

```

{"01110", "10001"},
{"01111", "10000"},
{"10000", "01111"},
{"10001", "01110"},
{"10010", "01101"},
{"10011", "01100"},
{"10100", "01011"},
{"10101", "01010"},
{"10110", "01001"},
{"10111", "01000"},
{"11000", "00111"},
{"11001", "00110"},
{"11010", "00101"},
{"11011", "00100"},
{"11100", "00011"},
{"11101", "00010"},
{"11110", "00001"},
{"11111", "00000"},
};

```

```

std::map<std::string, std::string> decodeTable{
    {"11111", "00000"},
    {"11110", "00001"},
    {"11101", "00010"},
    {"11100", "00011"},
    {"11011", "00100"},
    {"11010", "00101"},
    {"11001", "00110"},
    {"11000", "00111"},
}

```

```
    {"10111", "01000"},
    {"10110", "01001"},
    {"10101", "01010"},
    {"10100", "01011"},
    {"10011", "01100"},
    {"10010", "01101"},
    {"10001", "01110"},
    {"10000", "01111"},
    {"01111", "10000"},
    {"01110", "10001"},
    {"01101", "10010"},
    {"01100", "10011"},
    {"01011", "10100"},
    {"01010", "10101"},
    {"01001", "10110"},
    {"01000", "10111"},
    {"00111", "11000"},
    {"00110", "11001"},
    {"00101", "11010"},
    {"00100", "11011"},
    {"00011", "11100"},
    {"00010", "11101"},
    {"00001", "11110"},
    {"00000", "11111"},
};
```

```
//-----
std::string GenerateRandomString(const char *alphabet, int alphabetSize, int
strLen)
```

```

{
    std::string answer = "";
    answer.resize(strLen);

    for (int i = 0; i < strLen; i++)
    {
        answer[i] = alphabet[rand() % alphabetSize];
    }

    return answer;
}

//-----
void ExpandStrBySymbol(std::string &str, const char sym, int multLen)
{
    int diff = 0;
    if (str.size() < multLen)
    {
        diff = multLen - str.size();
    }
    else
    {
        int remainder = str.size() % multLen;
        diff = multLen - remainder;
    }

    if (diff == 0)
        return;

```

```

    std::string addStr = std::string(diff, sym);
    str = str + addStr;
    str.shrink_to_fit();
}

//-----
std::string CodeStr(std::string inputStr, int groupLen, const std::map<std::string,
std::string> &codeTable)
{
    std::string outputString = "";

    for (int i = 0; i < inputStr.size(); i += groupLen)
    {
        std::string groupStr = inputStr.substr(i, groupLen);

        std::string codeGroupStr = codeTable.at(groupStr);

        outputString += codeGroupStr;
    }

    return outputString;
}

//-----
bool Test(std::string inputStr, int groupLen, const std::map<std::string,
std::string> &encodeTable, const std::map<std::string, std::string>
&decodeTable)
{
    std::string outputStr = CodeStr(inputStr, groupLen, encodeTable);

```

```

std::string decodeStr = CodeStr(outputStr, groupLen, decodeTable);

bool isEqual = (inputStr == decodeStr);

const char *color = isEqual ? "\033[32m" : "\033[31m";
const char *reset = "\033[0m";
const char *status = isEqual ? "OK" : "Error";

    printf("%s%-7s%s  [%-20s]  [%-20s]  [%-20s]\n",color,  status,  reset,
inputStr.c_str(), outputStr.c_str(), decodeStr.c_str());

    return isEqual;
}

//-----

int main()
{
    srand(time(0));

    int strLen = 0;
    std::string inputStr = "";
    printf("%-7s %-20s %-20s %-20s\n", "Status", " InputStr", "  EncodeStr", "
DecodeStr");
    for (int i = 0; i < TEST_COUNT; i++)
    {
        strLen = rand() % MAX_LEN + 1;

        inputStr = GenerateRandomString(V, N, strLen);

```

```

ExpandStrBySymbol(inputStr, V[0], M);

bool isCorrect = Test(inputStr, M, encodeTable, decodeTable);
}
}

//-----

```

5. Пример открытого текста и соответствующей ему шифрограммы.

```

> /home/kukirmash/labs/7sem/ZI/lab1/build/lab1

```

Status	InputStr	EncodeStr	DecodeStr
OK	[00000]	[11111]	[00000]
OK	[10111000101101000000]	[0100011101001011111]	[10111000101101000000]
OK	[1000110010]	[0111001101]	[1000110010]
OK	[1001100000]	[0110011111]	[1001100000]
OK	[10000]	[01111]	[10000]
OK	[0111100000]	[1000011111]	[0111100000]
OK	[111101101011100]	[000010010100011]	[111101101011100]
OK	[01000]	[10111]	[01000]
OK	[0000000000]	[1111111111]	[0000000000]
OK	[11110]	[00001]	[11110]

6. Выводы по работе.

В ходе данной работы был изучен метод шифрования данных методом подстановки. Также были получены практические навыки использования данного метода.